Perception-Based Image Retrieval over Internet

Author list removed for double blind reviewing

Abstract

For image retrieval over the Internet, text-based search is not sufficient because an image query concept is often hard to articulate. In addition, articulation of an image can be subjective. Most users would find it hard to describe an image query in low-level perceptual features. We believe that one desirable paradigm for image search is to learn users' query concepts through *active learning*. A challenging constraint for this learning task is the scarcity of training data, since impatient users typically do not provide a great deal of feedback during a search. In this paper, we present a new image retrieval system that employs a new search paradigm called *perception-based image retrieval*. Under this paradigm, the query-concept learning problem is formulated as finding a binary classifier that separates relevant objects from those that are irrelevant to the query concept. We illustrate the flow of this new search scenario, describe the key components of the system and present two active learning algorithms used in the underlining engine. We further propose a hybrid learner based on these two algorithms which substantially improves the search quality of the individual algorithms. The experimental results demonstrate that the hybrid solution can quickly learn users' query concepts, and efficiently retrieve relevant images from large database. Once the user's query concept is captured, the traditional query-by-example scheme can be further employed for similarity search. Our extensive empirical study shows that even when the feature dimension is very high and target concepts are very specific, our system can grasp a complex query concept in a small number of user iterations.
Keywords:image retrieval, active learning, data mining, query concept, relevance feedback.

1 Introduction

For image-search tasks, a query concept is subjective and, therefore, hard to articulate. For instance, it is difficult to describe a desired image using low-level features such as color, shape, and texture. Different users may perceive the same image differently; and even if an image is perceived similarly, users may use different vocabulary (i.e., different combinations of low-level features) to depict it. Furthermore, most users are not trained to specify simple query criteria using formal query languages. It is thus necessary to build intelligent search engines that can quickly learn users' query concepts via *active learning* to capture subjective concepts and alleviate users from having to specify complex queries in their search.

Traditional active learning and relevance feedback techniques may not be suitable for online query-concept learning for at least two reasons. First, online users are typically impatient and cannot be expected to provide a great deal of feedback during the search. However, traditional data mining methods such as decision trees and neural networks require a large number of

training instances (i.e., samples) and can take a long time (more than a few seconds) to learn a concept [27]. Second, traditional relevance feedback methods [19, 31, 41] require users to provide good examples to seed a query. However, finding good seeds should be the job of the search engine itself, and this circular requirement leaves the core problem— learning users' query concepts—unsolved.

In this paper, we illustrate an image retrieval system based on a new perception-based search paradigm. Our system can help users of image content providers find the desired images much faster and much more easily. It can also help on-line shoppers browse and search products/auction items visually. Our system can be integrated seamlessly and synergistically with existing keyword-based image search to substantially shorten image seekers' search time and effort.

1.1 An Illustrative Example

We present an example produced from our image retrieval system to show the learning steps for grasping a query concept "sport hat". This interactive query session involves five screens that are illustrated in five figures. In this example, the image database is obtained from an e-commerce site.

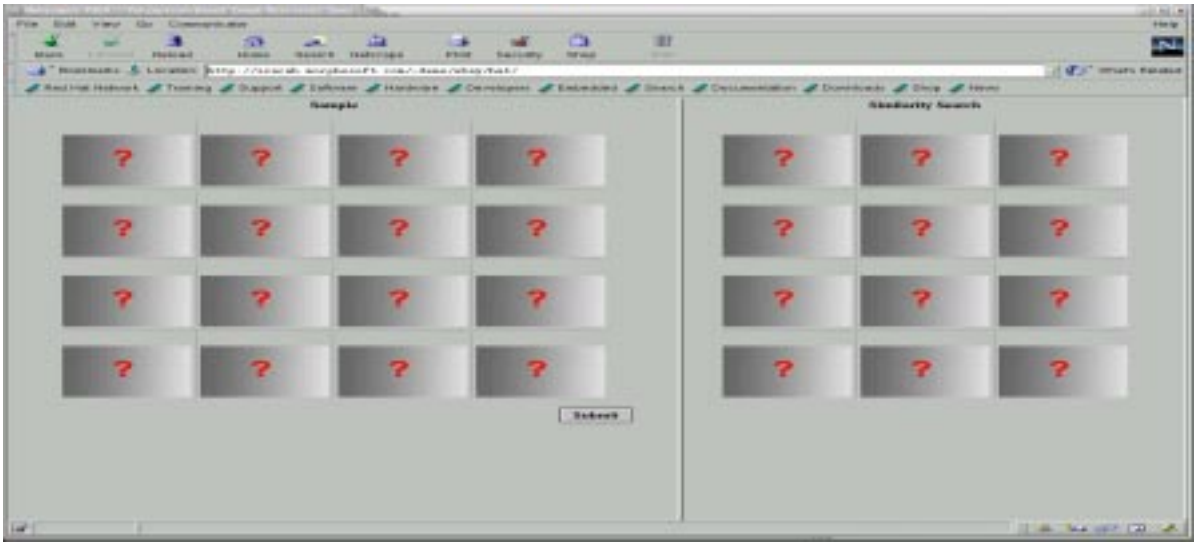


Figure 1: Sport Hat Query Screen #1.

- Screen 1. Initial Screen. The system presents the initial screen to the user as depicted in Figure 1. The screen is split into two frames vertically. On the left-hand side of the screen is the learner frame; on the right-hand side is the similarity search frame. Through the learner frame, the system learns what the user wants via an active learning process. The similarity search frame displays what the system thinks the user wants.

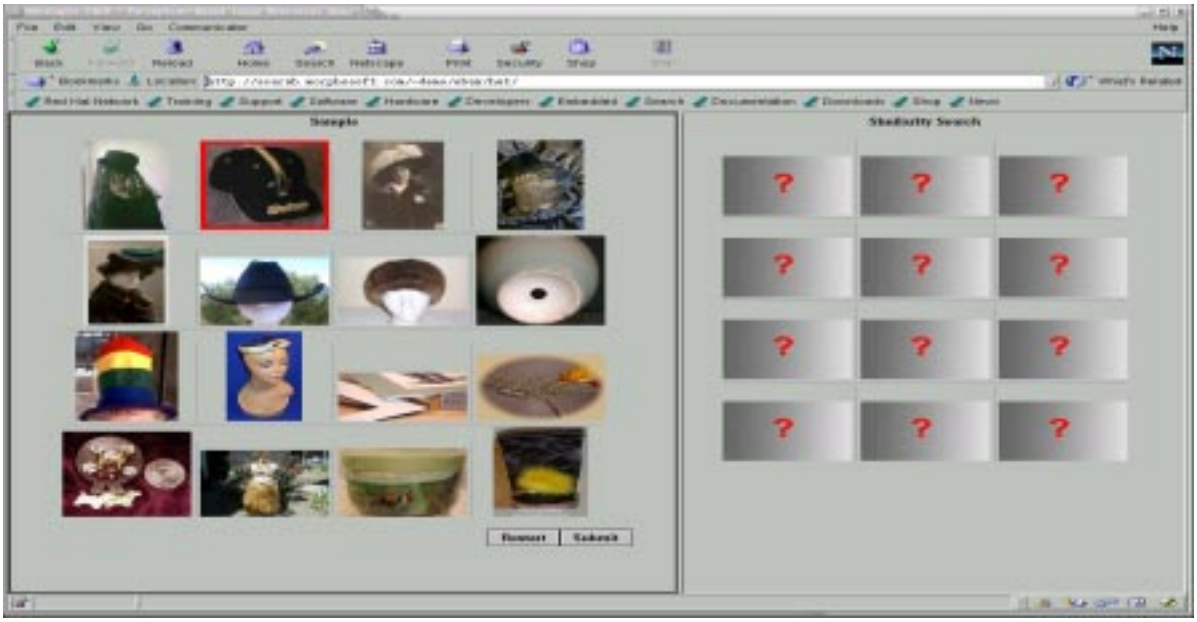


Figure 2: Sport Hat Query Screen #2.

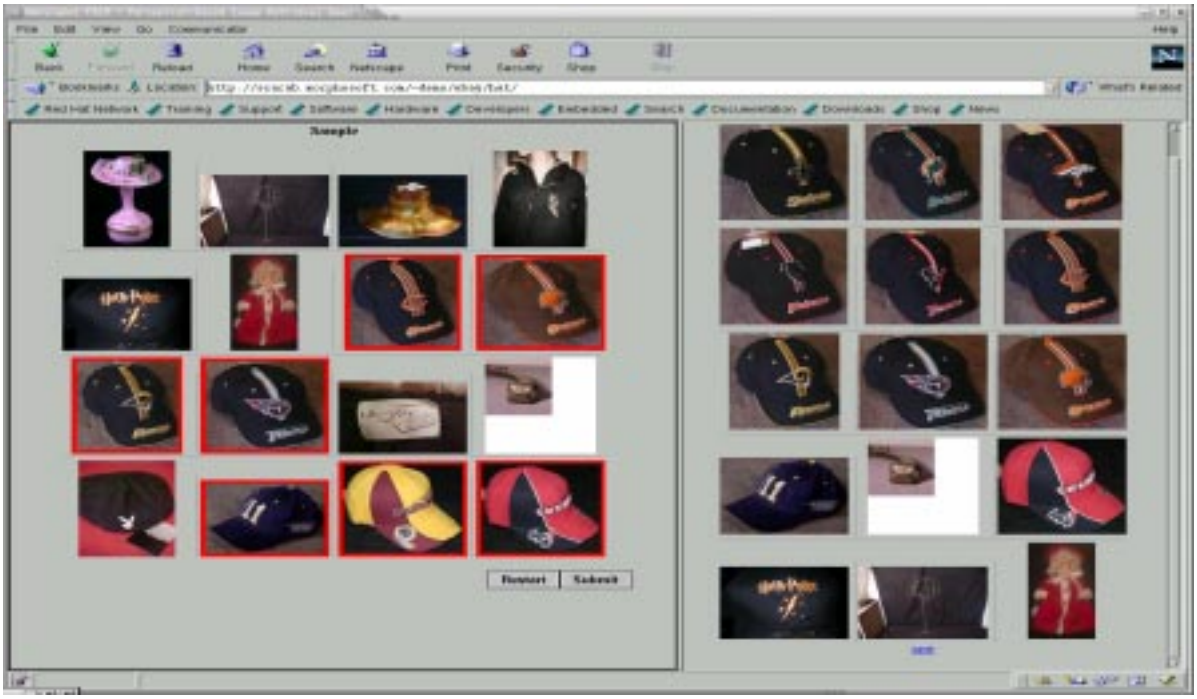


Figure 3: Sport Hat Query Screen #3.

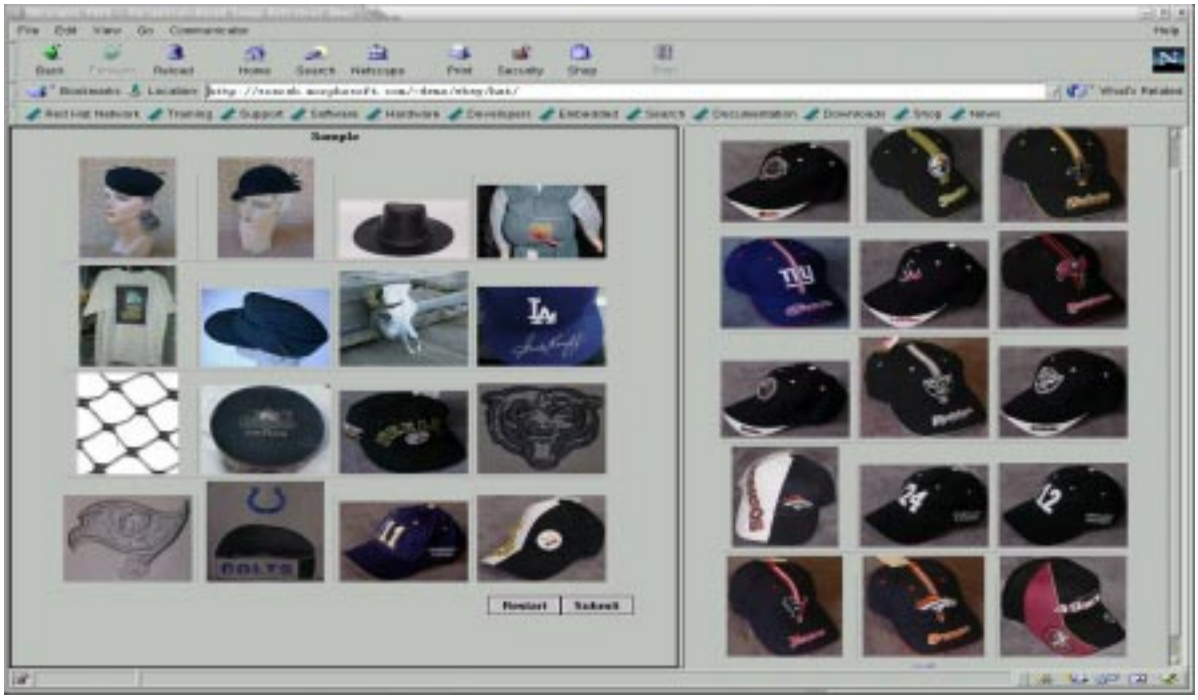


Figure 4: Sport Hat Query Screen #4.

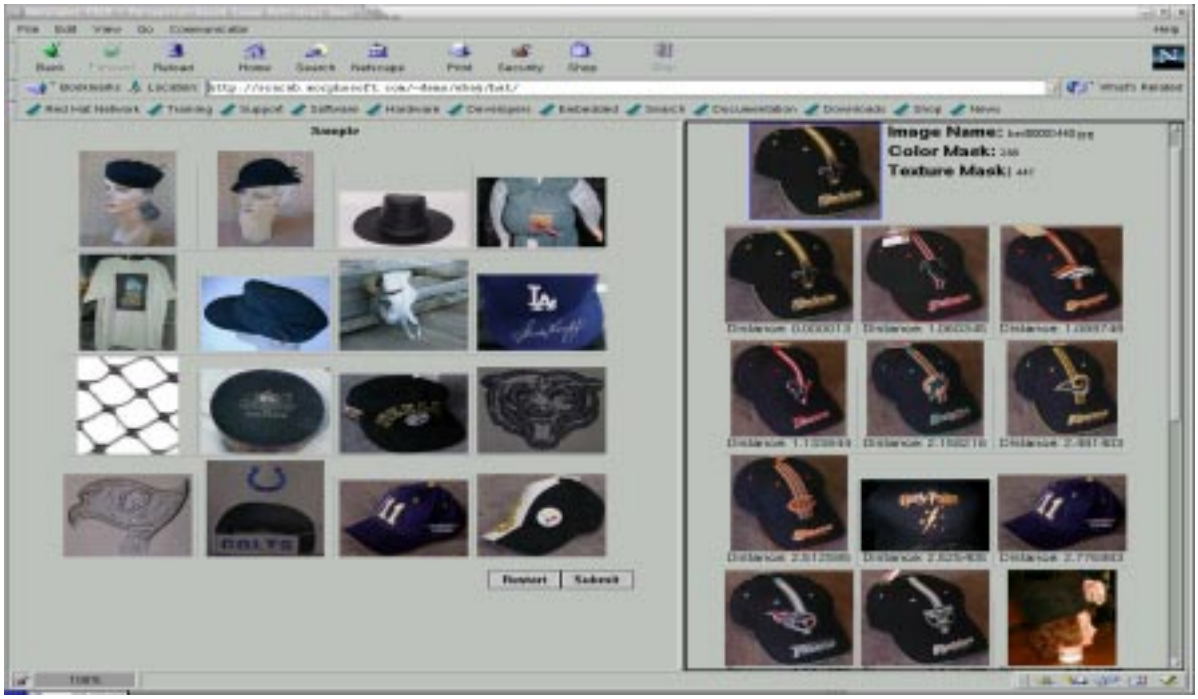


Figure 5: Sport Hat Similarity Query #5.

- Screen 2. Sampling and relevance feedback starts. Once the user clicks the "submit" button in the initial frame, the active learning step commences to learn what the user wants. The system presents a number of samples in the learner frame, and the user marks images that are relevant to his/her query concept by clicking on the relevant images. As shown in Figure 2, one image (the second image in the second row) are selected as relevant, and the rest of the unmarked images are considered irrelevant. The user indicates the end of his/her selection by clicking on the submit button in the learner screen. This action brings up the next screen.

- Screen 3. Sampling and relevance feedback continues. Figure 3 shows the third screen. First, the similarity search frame displays what the system thinks will match the user's query concept at this time. As the figure indicates, eleven returned images fit the concept of "sport hat." The user's query concept has been captured, though somewhat fuzzily. The user can ask the system to further refine the target concept by selecting relevant images in the learner frame. In this example, seven images (the last two images from the second row, the first two images from the third row, and the last three images from the last row) are relevant to the concept. After the user clicks on the submit button in the learner frame, the fourth screen is displayed.

- Screen 4. Sampling and relevance feedback ends. Figure 4 shows that all returned images in the similarity search frame fit the query concept (sport hat).

- Screen 5. Similarity search. At any time, the user can click on an image in the similarity search frame to request images that appear similar to the selected image. This step allows the user to zoom in onto a specific set of images that match some appearance criteria, such as color distribution, textures and shapes. As shown in Figure 5, after clicking on one of the hat images, the user will find similar hat images returned returned in the similarity search frame.

1.2 Proposed Image Retrieval Architecture

As illustated by the above exampl, our proposed system offers different browsing options. Users can query our system with either a concept or an example. For query-by-concept, our system is able to learn the concept via feedbacks from the users then retrieve relevant images from the database. For query-by-example, our system returns similar images to the example. Such a solution provides great browsing flexibility to users. For example, users could first use a concept to explore the image database, followed by query-by-example to further refine their search.

Figure 6 show the architecture of the proposed image retrieval system. It consists of the following components:

- *Image Database.* Our proposed system can be used to retrieve image for any image

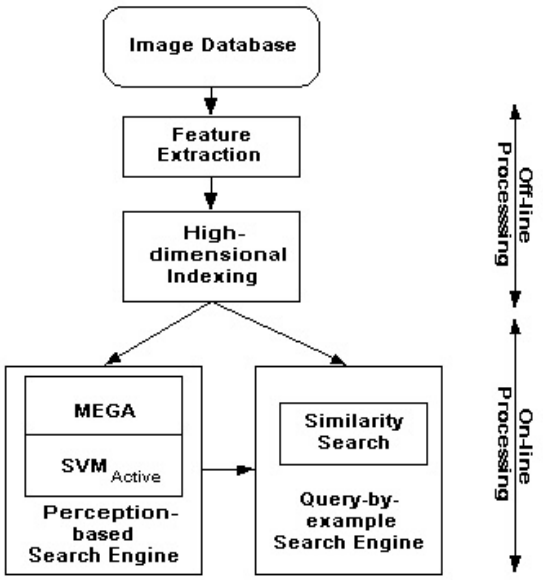


Figure 6: Proposed Image Retrieval Architecture.

database ranging from image content providers, e-commerce sites to corporation's and even individual's image collections.

- *Feature Extraction.* For each image in the database, we extract 144 low-level features (including color, shape, texture, etc) as the "signatures" of the image. The task of feature extraction is performed off-line and after such preprocessing, each image is characterized by its 144-dimension feature vector. Based on these feature vectors, our system can either distinguish one image from another or learn the common characteristics among a group of images.

- *High-dimensional Indexer.* The high-dimensional indexer provides fast access to image feature vectors for a large image database. Images feature vectors, that are close to each other in in the high-dimensional (144 dimensions) space, are grouped and stored together in the storage system. The high-dimensional indexer improves the scalability of the system in two aspects: fewer data retrievals from the storage system and shorter processing time. With the help of the indexer, the system can quickly decide whether an image group is relevant to the users' query concept. If not, there is no need to read in any image from the group and thus unnecessary retrieval of data is avoided and thus the run time is improved.

- *Perception-based Search Engine.* The perception-based search engine is an on-line image retrieval engine. Users can quickly personalize the engine and use the personalized engine to retrieve images. Here, we regard the problem of learning a query concept as learning a binary classifier that separates the images relevant to the query concept from the irrelevant ones. In this paper, we explore several active learning algorithms as the underlining

learning engine for image retrieval. In the later sections, we first present some details of two active learning algorithms, MEGA (The Maximizing Expected Generalization Algorithm) [1] and SVM$_{Active}$ (Support Vector Machine Active Learning) [2]. MEGA models a query concept in k-CNF, and at the same time, it uses k-DNF to bound the sampling space. MEGA can find positive examples quickly by judiciously selecting samples that can collect maximum information from users to refine both the concept and the sampling boundary. (We describe how MEGA works and contrast its sampling strategy with some traditional approaches in Section 2.) SVM$_{Active}$ is a margin-based active learning algorithm. When at least one positive and one negative example are identified, SVM$_{Active}$ can effectively refine the binary classifier, even if the class boundary is non-linear. (We describe how SVM$_{Active}$ works and its sampling strategies in Section 3.)

We further propose a pipeline-training technique which combines the strengths of MEGA (finding initial relevant objects to the concept) and SVM$_{Active}$ (refining the class boundary). The pipeline-training technique first uses MEGA to find initial positive samples and then it switches to SVM$_{Active}$ to refine the query concept. Our experimental results show that the pipeline-training technique can learn a concept substantially faster and more accurately than the individual algorithms can.

- *Query-by-example Similarity Search.* The similarity search engine queries the image database using an image selected by the user. Given a selected image, this engine finds all similar images based on some appearance criteria such as color, texture, and shape. One limitation of the query-by-example approach is that it cannot distinguish the desired subject from the backgrounds. Therefore, the engine might return irrelevant images that have the similar background as the selected image. This drawback can be overcome by the perception-based search engine described above. Therefore, this query-by-example engine is more suitable as a refinement tool after the query concept has been learned (through the perception-based search engine described above).

The proposed system has four major features that are crucial for effective and efficient image retrieval:

1. *Eliminating the seeding requirement.* Our learning system does not require to initiate a query with "good" seed images. Even if all initial samples presented to the user are negative, the negative-labeled instances can be used to shrink the pool of the next sampling round substantially. Therefore, the probability that a relevant example will be presented in the next iteration will be substantially higher.

2. *Supporting flexible concept formulation.* One can search for a general animal concept or a specific kind of sport hat (e.g., black sport hat). For example, if we mark only black sport hat images as relevant, the final result would show black sport hat images only. Also

note in the second screen that the system presents hats of different kinds, and in different shape, and on different backgrounds to ask for feedback. The system can intelligently explore more possibilities to refine the query concept.

3. *Accomplishing the above tasks quickly and accurately.*

As the perception-based image retrieval engine is the key and enabling element of the system, the rest of the paper will mainly focus this engine. We will discuss several alternative algorithms and implementations of this engines. In Section 2, we first present the algorithm of MEGA. Section 3 depicts the SVM_{Active} algorithm. Section 4 presents a combined learning strategy. The hybrid approach first uses MEGA to find relevant samples to initialize query-concept learning. It then uses SVM_{Active} to refine the query concept. Sections 5 and 6 report experimental results. Section 7 discusses related work. Finally, we offer our concluding remarks in Section 8.

2 MEGA

The first challenge of query concept learning is to find some relevant objects so that the concept boundary can be fuzzily identified. Finding a relevant object can be difficult if only a small fraction of the dataset satisfies the target concept. For instance, suppose the desired objects in a one-million-image dataset is 100 (0.1%). If we randomly select 20 objects per round for users to identify relevant objects, the probability of finding a positive sample after five rounds of random sampling is just 10%—clearly not acceptable.

We can improve the odds with an intelligent sampling method MEGA (The Maximizing Expected Generalization Algorithm), which finds relevant samples quickly, to initialize query-concept learning.

MEGA models query concepts in k-CNF [20], which can formulate virtually all practical query concepts[1]. MEGA uses k-DNF to bound the sampling space from which to select the most informative samples for soliciting user feedback.

Definition 1: **k-CNF**: For constant k, the representation class k-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each c_i is a disjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. (A Boolean variable represents an image feature.) No prior bound is placed on θ.

Definition 2: **k-DNF**: For constant k, the representation class k-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each d_i is a conjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

MEGA initializes the query concept-space (QCS) as a k-CNF and the candidate concept-

[1] k-CNF is more expressive than k-term DNF, and it has both polynomial sample complexity and time complexity [21, 26].

space (CCS) as a k-DNF. The QCS starts as the most specific concept and the CCS as the most general concept. The target concept is more general than the initial QCS and more specific than the initial CCS. The learner learns the QCS, while at the same time refining the CCS to delimit the boundary of the sampling space. (The shaded area in Figure 7 shows the sampling space that is between the QCS and the CCS.)

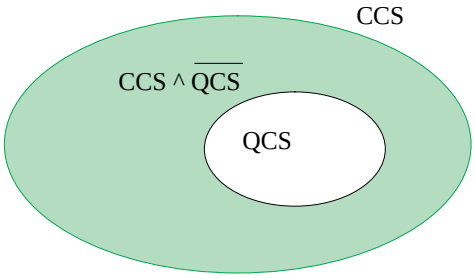


Figure 7: MEGA's Sampling Space: $CCS \wedge \overline{QCS}$.

To learn a target concept, the well-known Valiant's algorithm [38] uses positive-labeled instances to refine QCS and negative-labeled instances to refine CCS. The key difference between Valiant's algorithm and MEGA lies in how training samples are selected. Valiant's algorithm employs a random sampling strategy. MEGA makes sure that each sample can be most useful, and hence reduces the number of samples needed to learn a concept by employing two strategies.

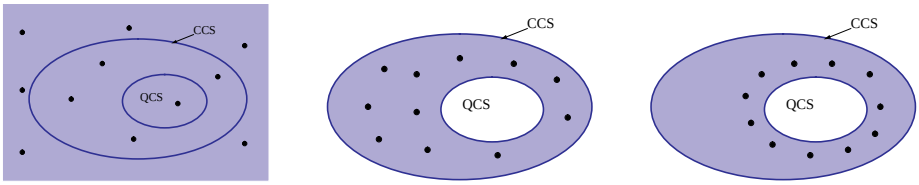
1. Bounding the sample space: Avoid choosing useless unlabeled instances by using the CCS and QCS to delimit the sampling boundary.

2. Maximizing the usefulness of a sample: Choose an example that will remove the maximum expected number of disjunctive terms from QCS. In other words, we choose an example that can maximize the expected generalization of the concept. Even if the example is labeled negative by the user, it can be useful to remove conjunctive terms in the CCS.

It may appear that if we pick an example that has more dissimilar disjunctions (compared to the QCS), we would have a better chance of eliminating more disjunctive terms. This is, however, not true. An example must be labeled by the user as positive to be useful for refining QCS. Unfortunately, an example is less likely to be labeled positive when it has more disjunctions that are dissimilar to the target concept. Therefore, there is a tradeoff between choosing an example that has more contradictory terms and choosing one that is more likely to be labeled positive.

Figure 8 shows how sampling algorithms can affect convergence speed.

- *Random*: Samples are randomly selected from the bulk of the domain (Figure 8(a)). Clearly, the samples that are more general than CCS or more specific than QCS cannot contribute to refining neither QCS nor CCS. In other words, random sampling can produce useless samples for learning a concept, and hence increase the total number of samples required.



(a) Random (b) B-Random (c) MEGA

Figure 8: Sampling Schemes.

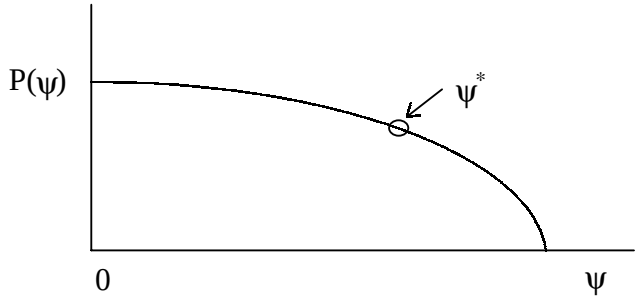


Figure 9: Sampling Tradeoffs.

- *Bounded Random*: Samples are randomly selected from between QCS and CCS (Figure 8(b)). This method is a logical improvement over random sampling, but it is not good enough.
- MEGA: Samples are selected from between QCS and CCS to eliminate the maximum expected number of terms (Figure 8(c)). Below, we describe how samples are selected by MEGA.

Let Ψ denote the number of disjunctions remaining in the concept. The number of disjunctions that can be eliminated in the current round of sampling, denoted as ψ, is between zero and Ψ. We can write the probability of eliminating ψ terms as $P_e(\psi)$. Figure 9 depicts another view of the effect of different sampling methods. The x-axis denotes the number of contradictory terms ψ in a sample compared to QCS. The y-axis shows $P_e(\psi)$, the probability that a sample is labeled positive by the user. $P_e(\psi)$ is a monotonically decreasing function of ψ. When ψ is large, the sample is likely to be useless, since it is unlikely to be marked positive. When ψ is small, the sample is likely to be useful, but the number of terms in QCS that the sample can eliminate is minimum. MEGA attempts to find the ψ that can eliminate the maximum expected number of disjunctive terms in QCS. Theorem 1 shows how the optimal ψ is computed. The detailed MEGA algorithm is documented in [1].

[Theorem 1] Let Ψ denote the number of disjunctions remaining in the concept. Let random variable Φ_i be 1 if term T_i is in the concept and 0 otherwise. Assume that the Φ_i's are iid (independent and identically distributed), and the probability of Φ_i being 1 is p $(0 < p < 1)$.

the optimal ψ value, denoted by ψ^* is

$$\psi^* = \Psi, \; if \; \frac{1}{ln\frac{1}{1-p}} > \Psi, \quad \psi^* = \frac{1}{ln\frac{1}{1-p}}, \; otherwise.$$

[**Proof**] Please refer to [1] for the proof.

3 SVM_{Active}

Once some relevant and some irrelevant samples are marked, we can employ SVM_{Active} to refine the class boundary. SVMs have strong theoretical foundations and excellent empirical successes. They have been applied to tasks such as handwritten digit recognition [39], object recognition [30], and text classification [17]. We added the active learning component to SVMs for selecting the most informative samples to query a user and to quickly refine a boundary that separates data objects that satisfy the user's query concept from the rest of the dataset.

For the purpose of query-concept learning, we consider SVMs in the binary classification setting. We are given training data $\{\mathbf{x_1} \ldots \mathbf{x_n}\}$ that are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^{\lceil}$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin. All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space $\mathcal{X}$ to a higher dimensional feature space $\mathcal{F}$ via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{K}(\mathbf{x_i}, \mathbf{x})$. When $f(\mathbf{x}) \geq \mathbf{0}$ we classify $\mathbf{x}$ as $+1$, otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [5] we can write: $K(\mathbf{u}, \mathbf{v}) = \mathbf{\Phi}(\mathbf{u}) \cdot \mathbf{\Phi}(\mathbf{v})$ where $\Phi : \mathcal{X} \rightarrow \mathcal{F}$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{\Phi}(\mathbf{x_i}). \tag{1}$$

Thus, by using K we are implicitly projecting the training data into a different (often higher dimensional) feature space $\mathcal{F}$. The SVM then computes the α_is that correspond to the maximal margin hyperplane in $\mathcal{F}$. By choosing different kernel functions, we can implicitly project the training data from $\mathcal{X}$ into spaces $\mathcal{F}$ for which hyperplanes in $\mathcal{F}$ correspond to more complex decision boundaries in the original space $\mathcal{X}$.

Intuitively, SVM_{Active} works by combining the following three ideas:

1. SVM_{Active} regards the task of learning a target concept as one of learning an SVM binary classifier. An SVM captures the query concept by separating the relevant images from the irrelevant images with a hyperplane in a projected space, usually a very high-dimensional

one. The projected points on one side of the hyperplane are considered relevant to the query concept and the rest irrelevant.

2. SVM_{Active} learns the classifier quickly via active learning. The active part of SVM_{Active} selects the most informative instances with which to train the SVM classifier. This step ensures fast convergence to the query concept in a small number of feedback rounds.

3. Once the classifier is trained, SVM_{Active} returns the top-k most relevant images. These are the k images farthest from the hyperplane on the query concept side.

Let $SetV$ denote the uncertain concept space. The current version of SVM_{Active} employs Tong & Koller's lemma [37], which chooses a pool-query that halves the uncertain space. In reality, however, finding the pool-query that can divide the uncertain space in half can be difficult and computationally intensive. Given an unlabeled instance $\mathbf{x}$ from the pool, it is not practical to explicitly compute the sizes of the new uncertain spaces $\mathcal{V}^-$ and $\mathcal{V}^+$ (i.e., the uncertain spaces obtained when $\mathbf{x}$ is labeled as -1 and $+1$ respectively). We have tried a simple method that works as follows: Learn an SVM with the existing labeled data, and choose as the next batch of samples, those closest to the hyperplane in feature space $\mathcal{F}$.

The initial theorem of Tong & Koller assumes one sample is chosen for each user iteration. For relevance feedback, we have a need for performing multiple pool-queries at the same time. It is not practical to present one data object at a time for the user to label — the user is likely to quickly lose patience after a few rounds of pool-querying. To prevent this from happening we would like to present the user with multiple examples (say, sixteen) at each round of pool-querying. Thus, for each round, the active learner has to choose not just one example to be labeled but sixteen. Theoretically it would be possible to consider the size of the resulting uncertain spaces for each possible labeling of each possible set of twenty pool-queries, but clearly this is impractical. Thus instead, for matters of computational efficiency, we use the simple approach of choosing pool-queries the sixteen instances closest to the separating hyperplane.

Once we have performed a number of rounds of querying, we will have amassed a small set of labeled instances. We learn a final SVM using these labeled instances and use this SVM as our final classifier. In the retrieval task, we wish to return instances in order of decreasing relevance. This is easy to achieve with an SVM. Recall that an SVM is a hyperplane in the feature space such that any feature vector lying on one side of the hyperplane is labeled as "relevant" and any feature vector lying on the opposite side of the hyperplane is labeled as "not relevant." We have greatest confidence in those labels when the vectors are farthest away from the hyperplane. Hence, we can rank all of the instances in the database in order of relevance, simply by computing their distance from the hyperplane.

4 Hybrid Algorithms

Although MEGA and SVM$_{Active}$ are effective active learning algorithms, we believe combining their strengths can result in even better learning algorithms. In this section we propose a hybrid algorithm.

As discussed in Section 3, the SVM$_{Active}$ scheme needs at least one positive and one negative example to start. MEGA is not restricted by this seeding requirement, and it is able to find relevant examples quickly by refining the sampling boundary. It is therefore logical to employ MEGA to perform the *initialization* task. Once some relevant images are found, the refinement step can be executed by either MEGA or SVM$_{Active}$.

Thus, we can have three execution alternatives.

1. MEGA only. Use MEGA all the way to learn a concept.
2. SVM$_{Active}$ only. Use random sampling to find the first relevant example(s) and then use SVM$_{Active}$ to learn a concept.
3. Pipeline learning. Use MEGA to find initial relevant objects, and then switch to SVM$_{Active}$ for refining the binary classifier and ranking returned objects.

5 Experiments

We implemented both MEGA and SVM$_{Active}$ in C, $C++$, and tested using an Intel Pentium III^{TM} workstation running Linux. We have implemented an industrial strength prototype with all features discussed in this paper.

We tested our prototype with the intent of answering four central questions.

1. Do MEGA and SVM$_{Active}$ learn to model complex query concepts in very high dimensional spaces both quickly and with a small number of iterations?
2. Does using MEGA to find the first relevant object(s) and then switching to SVM$_{Active}$ a more effective algorithm than using the two learning algorithms individually?
3. Does the percentage of relevant data affect learning performance? (If we reduce the matching data from 5% of the dataset to 1%, does it take more iterations to learn query concepts?)

5.1 Data

For our empirical evaluation of our learning methods we used a real-world image dataset: a twenty-category image dataset where each category consisted of 200 to 300 images. Images for

Filter Name	Resolution	Representation
Masks	Coarse	Appearance of culture colors
Spread	Coarse	Spatial concentration of a color
Elongation	Coarse	Shape of a color
Histograms	Medium	Distribution of colors
Average	Medium	Similarity comparison within the same culture color
Variance	Fine	Similarity comparison within the same culture color

Table 1: Multi-resolution Color Features.

this dataset were collected from Corel Image CDs and the Internet.

- *Twenty-category* set. The set consists of images of *architecture*, *bears*, *cities*, *clouds*, *couples*, *flowers*, *insects*, *ladies*, *landscape*, *objectionable images*, *planets*, *tigers*, *tools*, *waves*, *elephants*, *fabrics*, *fireworks*, *food*, and *textures*.

To enable an objective measure of performance, we assumed that a query concept was an image category, or a superset or a subset of a category. For instance, a query can be "wild animals" including "tigers", "elephants", and "bears." A query can be "white bears" (a subset of "bears"), or "purple flowers" (a subset of "flowers"). The learners do not have prior knowledge about image categories. Unlike some recently developed systems [40] that contain a semantical layer between image features and queries to assist query refinement, our system does not have an explicit semantical layer. We argue that having a hard-coded semantical layer can make a retrieval system restrictive to some predefined concepts. Rather, dynamically learning the semantics of a query concept is more flexible and hence makes the system more useful.

Our algrithms treat each image as a 144-dimension vector. We characterize images by two main features: color and texture. We consider shape as an attribute of these main features.

- *Color.* Although the wavelength of visible light ranges from 400 nanometers to 700 nanometers, research [11] shows that the colors that can be named by all cultures are generally limited to eleven. In addition to *black* and *white*, the discernible colors are *red*, *yellow*, *green*, *blue*, *brown*, *purple*, *pink*, *orange* and *gray*.

  We first divide color into 12 color bins including 11 bins for culture colors and one bin for outliers [14]. At the coarsest resolution, we characterize color using a color mask of 12 bits. To record color information at finer resolutions, we record eight additional features for each color. These eight features are color histograms, color means (in H, S and V channels), color variances (in H, S and V channel), and two shape characteristics: elongation and spreadness. Color elongation characterizes the shape of a color and spreadness characterizes how that color scatters within the image [22]. Table 1 summarizes color features in coarse, medium and fine resolutions.

- *Texture.* Texture is an important cue for image analysis. Studies [24, 35, 36, 23] have shown that characterizing texture features in terms of structuredness, orientation, and scale (coarseness) fits well with models of human perception. A wide variety of texture analysis methods have been proposed in the past. We choose a discrete wavelet transformation (DWT) using quadrature mirror filters [35] because of its computational efficiency.

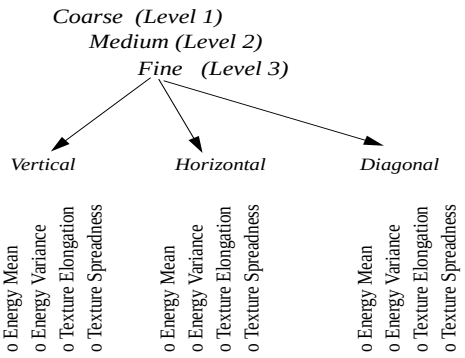


Figure 10: Multi-resolution Texture Features.

Each wavelet decomposition on a 2-D image yields four subimages: a $\frac{1}{2} \times \frac{1}{2}$ scaled-down image of the input image and its wavelets in three orientations: horizontal, vertical and diagonal. Decomposing the scaled-down image further, we obtain the tree-structured or wavelet packet decomposition. The wavelet image decomposition provides a representation that is easy to interpret. Every subimage contains information of a specific scale and orientation and also retains spatial information. We obtain nine texture combinations from subimages of three scales and three orientations. Since each subimage retains the spatial information of texture, we also compute elongation and spreadness for each texture channel. Figure 10 summarizes texture features.

5.2 Queries

We separate our queries into two categories: 5% and 1% queries. The matching images for each of the more specific query concepts such as "purple flowers" and "white bears" account for about 1% of the total dataset. More general concepts such as "bears", "flowers" and "architectures" have about 5% matching images in the dataset. For each experiment, we will report results for these two categories of queries separately.

5.3 Evaluations

The goal of our learning algorithms is to learn a given concept through a relevance feedback process. In this process, at each feedback round we select ten images to ask the user to label

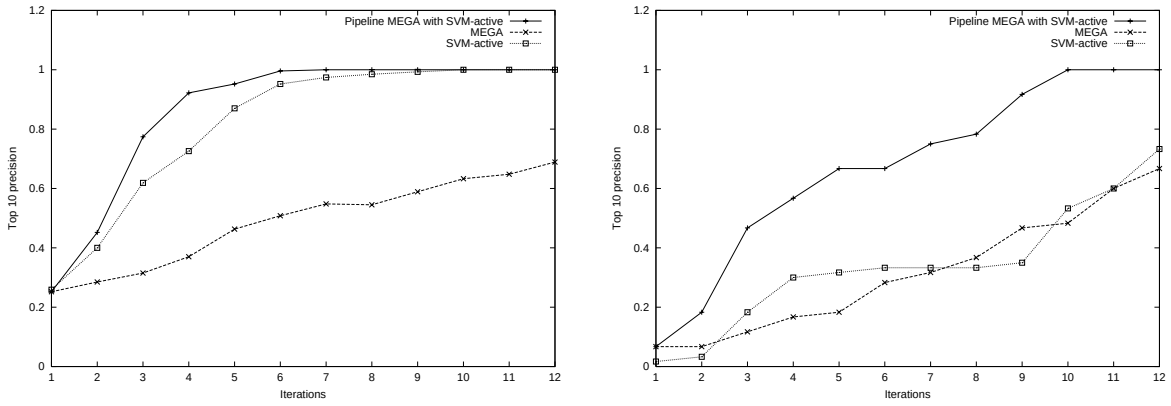


Figure 11: Precision versus iterations for top-10 retrieval (a) 5% and (b) 1% queries.

as "relevant" or "not relevant" with respect to the query concept. (Ten images are chosen because they can comfortably fit onto the screen.) The learners then use the labeled instances to successively refine the concept boundary. After the relevance feedback rounds have finished the learning algorithms retrieve the top-k most relevant images from the dataset based on the final concept learned. Accuracy is then computed by looking at the fraction of the k returned result that belongs to the target image category. Notice that this is equivalent to computing the precision on the top-k images. This measure of performance appears to be the most appropriate for the image retrieval task — particularly since, in most cases, not all of the relevant images can to be displayed to the user on one screen. As in the case of web searching, we typically wish the first screen of returned images to contain a high proportion of relevant images. We are less concerned that not every single instance that satisfies the query concept is displayed. (In other words, we are concerned more with precision than with recall.)

In the experiment, we report precisions of the top-10 and top-20 retrievals to measure performance. To be objective, we run each experiment ten times and report the average precisions.

6 Results and Discussion

Here we present our analysis of the results, organized with regard to the central questions listed at the start of Section 5.

6.1 MEGA, SVM$_{Active}$, and Pipelining MEGA with SVM$_{Active}$

For top-10 retrieval with 5% matching data (Figure 11(a)), SVM$_{Active}$ clearly outperforms MEGA. The major weakness of SVM$_{Active}$ is in initialization—finding the first few positive samples. Such weakness may not seriously affect this experiment, since there is a high probabil-

ity of finding one of the 5% positive examples through random sampling. For queries with only 1% matching data (Figure 11(b)), such a weakness becomes more significant, because it substantially degrades SVM$_{Active}$'s performance. Especially, its precision in the first two iterations is very low. Overall, the precision of MEGA and SVM$_{Active}$ is similar for 1% queries.

The hybrid algorithm (pipelining MEGA with SVM$_{Active}$) clearly outperforms both SVM$_{Active}$ and MEGA. The difference in precision is more significant for queries with 1% matching data than for those with 5% matching data. This trend indicates the strength of the hybrid algorithm in handling more specific query concepts and/or larger datasets. Note that the precision of the hybrid algorithm reaches 100% after 6 and 10 iterations, respectively, for these two experiments (Figures 11(a) and (b)).

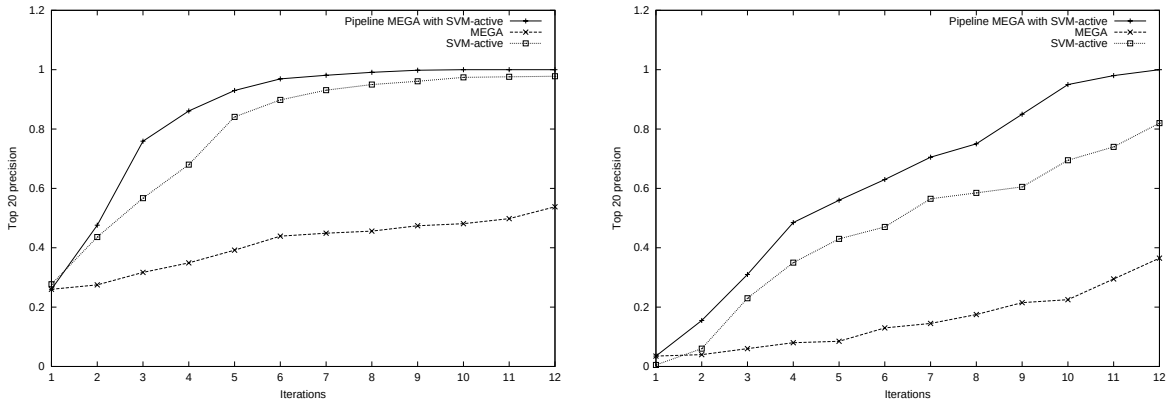


Figure 12: Precision versus iterations for top-20 retrieval (a) 5% and (b) 1% queries.

For top-20 retrieval (Figures 12(a) and (b)), the precision of the hybrid algorithm remains the highest, followed by SVM$_{Active}$ and MEGA. Again, the differences in their performances are more significant for queries with 1% matching data than for those with 5% matching data. For 1% queries, the hybrid algorithm achieves 100% precision after 12 iterations, which indicates the robustness of this new enhancement.

6.2 Observations

Our experiments have answered the four questions that we stated in the beginning of Section 5.

1. MEGA and SVM$_{Active}$ can learn complex query concepts in high-dimensional spaces in a small number of user iterations. Compared to some recent schemes proposed by the database community such as Mindreader [15] and Falcon [41], MEGA and SVM$_{Active}$ perform substantially better.

2. The hybrid scheme that uses MEGA to find the first relevant object(s) and then switches to SVM$_{Active}$ works significantly better than using the two learning algorithms individually.

3. When the matching data is scarce, the number of iterations required to learn a concept increases.

7 Related Work

Of late, ensemble techniques such as *bagging* [3], *arcing* [4], and *boosting* [33, 13] have been proposed to improve learning accuracy for decision trees and neural networks. These ensemble schemes enjoy success in improving classification accuracy through bias or variance reduction, but they do not help reduce the number of samples required to learn a query concept. In fact, most ensemble schemes actually increase learning time because they introduce learning redundancy for improving prediction accuracy [8, 13, 16, 28].

To reduce the number of required samples, the machine learning community have conducted several studies of active learning for classification. The *query by committee* algorithm [34, 10] uses a distribution over all possible classifiers and attempts to greedily reduce the entropy of this distribution. This general purpose algorithm has been applied in a number of domains[2] using classifiers (such as Naive Bayes classifiers [7, 25]) for which specifying and sampling classifiers from a distribution is natural. Probabilistic models such as the Naive Bayes classifier provide interpretable models and principled ways to incorporate prior knowledge and data with missing values. However, they typically do not perform as well as discriminative methods such as SVMs [17, 9].

Relevance feedback techniques proposed by the information retrieval and database communities also perform non-random sampling. The study of [32] puts these query refinement approaches into three categories: *query reweighting*, *query point movement* and *query expansion*.

- *Query reweighting* and *query point movement* [15, 29, 31]. Both query reweighting and query point movement use nearest-neighbor sampling: They return top ranked objects to be marked by the user and refine the query based on the resulting feedback. These methods tend to require a large number of image instances to achieve any substantial refinement since they do not tend to explore the image space aggressively.
- *Query expansion* [32, 41]. The *query expansion* approach can be regarded as a multiple-instances sampling approach. The samples of the next round are selected from the neighborhood (not necessarily the nearest ones) of the positive-labeled instances of the previous round. The study of [32] shows that query expansion achieves only a slim margin of improvement (about 10% in precision/recall) over query point movement.

For image retrieval, the PicHunter system [6] uses Bayesian prediction to infer the goal image, based upon the users' actions. The system shows that employing active learning can drastically

[2] Although, to our knowledge, not to the image retrieval domain.

cut down the number of iterations (up to 80% in some experiments). However the authors also point out that their scheme is computationally intensive, since it needs to recompute the conditional probability for all unlabeled samples after each round of user feedback and hence may not scale well with larger dataset sizes.

8 Conclusion

This paper proposes an image retrieval system that uses active learning to capture complex and subjective query concepts. We have proposed using MEGA followed by SVM_{Active} as our perception-based search engine. Our system uses MEGA to quickly find objects relevant to the query concept, then switches to SVM_{Active} once relevant objects are found. Our experimental results show that this hybrid approach outperforms MEGA and SVM_{Active} when they are used individually.

Unlike traditional data mining tasks where training data may be abundant, in an online training task, training data may be meager. We believe that being able to learn complex and subjective query concepts despite restrictions of time and meagerness of samples, can make a search engine more user-friendly and personalized. Many applications can benefit from this technology. In e-commerce application it could help customers quickly locating the desired products; in crime fighting it could help with identification; in medicine it could help distinguish abnormalities.

References

[1] Reference hidden for double blind reviewing.

[2] Reference hidden for double blind reviewing.

[3] L. Breiman. Bagging predicators. *Machine Learning*, pages 123–140, 1996.

[4] L. Breiman. Arcing classifiers. *The Annals of Statistics*, pages 801–849, 1998.

[5] C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

[6] I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.

[7] I. Dagan and S. Engelson. Committee-based sampling for training probabilistic classifiers. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 150–157. Morgan Kaufmann, 1995.

[8] T. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artifical Intelligence Research*, 2, 1995.

[9] S. Dumais, J. Platt, D. Heckerman, and M. Sahami. Inductive learning algorithms and representations for text categorization. In *Proceedings of the Seventh International Conference on Information and Knowledge Management*. ACM Press, 1998.

[10] Y. Freund, H. Seung, E. Shamir, and N. Tishby. Selective sampling using the Query by Committee algorithm. *Machine Learning*, 28:133–168, 1997.

[11] E. B. Goldstein. *Sensation and Perception (5th Edition)*. Brooks/Cole, 1999.

[12] T. Graepel, R. Herbrich, and K. Obermayer. Bayesian transduction. *Advances in Neural Information Systems Processing*, 12, 2000.
[13] A. Grove and D. Schuurmans. Boosting in the limit: Maximizing the margin of learned ensembles. In *Proc. 15th National Conference on Artificial Intelligence (AAAI)*, 1998.
[14] K. A. Hua, K. Vu, and J.-H. Oh. Sammatch: A flexible and efficient sampling-based image retrieval technique for image databases. *Proceedings of ACM Multimedia*, November 1999.
[15] Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.
[16] G. James and T. Hastie. *Error Coding and PaCT's*. 1997.
[17] T. Joachims. Text categorization with support vector machines. In *Proceedings of the European Conference on Machine Learning*. Springer-Verlag, 1998.
[18] T. Joachims. Transductive inference for text classification using support vector machines. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 200–209. Morgan Kaufmann, 1999.
[19] K. S. Jones and P. W. (Editors). *Readings in Information Retrieval*. Morgan Kaufman, July 1997.
[20] M. Kearns, M. Li, and L. Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.
[21] M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.
[22] J.-G. Leu. Computing a shape's moments from its boundary. *Pattern Recognition*, pages Vol.24, No.10,pp.949–957, 1991.
[23] W. Y. Ma and H. Zhang. Benchmarking of image features for content-based retrieval. *Proceedings of Asilomar Conference on Signal, Systems & Computers*, 1998.
[24] B. Manjunath, P. Wu, S. Newsam, and H. Shin. A texture descriptor for browsing and similarity retrieval. *Signal Processing Image Communication*, 2001.
[25] A. McCallum and K. Nigam. Employing EM in pool-based active learning for text classification. In *Proceedings of the Fifteenth International Conference on Machine Learning*. Morgan Kaufmann, 1998.
[26] T. Michell. *Machine Learning*. McGraw Hill, 1997.
[27] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
[28] M. Moreira and E. Mayoraz. Improving pairwise coupling classification with error correcting classifiers. *Proceedings of the Tenth European Conference on Machine Learning*, April 1998.
[29] M. Ortega, Y. Rui, K. Chakrabarti, A. Warshavsky, S. Mehrotra, and T. S. Huang. Supporting ranked boolean similarity queries in mars. *IEEE Transaction on Knowledge and Data Engineering*, 10(6):905–925, December 1999.
[30] C. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In *Proceedings of the International Conference on Computer Vision*, 1998.
[31] K. Porkaew, K. Chakrabarti, and S. Mehrotra. Query refinement for multimedia similarity retrieval in mars. *Proceedings of ACM Multimedia*, November 1999.
[32] K. Porkaew, S. Mehrota, and M. Ortega. Query reformulation for content based multimedia retrieval in mars. *ICMCS*, pages 747–751, 1999.
[33] R. Schapire, Y. Freund, P. Bartlett, and W. Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceeding of the Fourteenth International Conference on Machine Learning*. Morgan Kaufmann, 1997.
[34] H. Seung, M. Opper, and H. Sompolinsky. Query by committee. In *Proceedings of the Fifth Workshop on Computational Learning Theory*, pages 287–294. Morgan Kaufmann, 1992.
[35] J. Smith and S.-F. Chang. Automated image retrieval using color and texture. *IEEE Transaction on Pattern Analysis and Machine Intelligence*, November 1996.
[36] H. Tamura, S. Mori, and T. Yamawaki. Texture features corresponding to visual perception. *IEEE Transaction on Systems Man Cybernet (SMC)*, 1978.
[37] S. Tong and D. Koller. Support vector machine active learning with applications to text classification. *Proceedings of the 17th International Conference on Machine Learning*, pages 401–412, June 2000.
[38] L. Valiant. A theory of learnable. *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, pages 436–445, 1984.

[39] V. Vapnik. *Statistical Learning Theory.* Wiley, 1998.

[40] J. Wang, J. Li, and G. Wiederhold. Simplicity: Semantics-sensitive integrated matching for picture libraries. *ACM Multimedia Conference*, 2000.

[41] L. Wu, C. Faloutsos, K. Sycara, and T. R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The* 26th *VLDB Conference*, September 2000.